Rapport de mi parcours

Titre : Éditeur de maillage 3D interactif

Auteurs : DIAKITE Ibrahima Sory Tania

Dépôt : https://github.com/D-Ibrahima/Projet_Halfedge

Introduction :

Ce projet, réalisé dans le cadre du module C++ & Software Engineering (Université Bourgogne Europe, année 2025–2026), consiste à concevoir en C++ moderne un éditeur de maillage 3D interactif. Un maillage triangulaire est un graphe plongé dans R^3 : ses sommets portent des positions 3D et ses arêtes décrivent la connectivité de la surface. Pour manipuler cette connectivité efficacement, nous nous appuyons sur la structure Half-Edge canonique, où chaque demi-arête ne stocke que quatre pointeurs (target, face, next, twin). Elle permet de parcourir les voisinages et, à terme, d'appliquer des modifications topologiques en temps réel : insertion de sommet, découpe d'arête, suppression de face.

Ce rapport de mi-parcours présente l'avancement du projet. Il fait d'abord le point sur chaque jalon, puis explique la structure Half-Edge, l'architecture logicielle et le travail réalisé dans le code. Il détaille ensuite la planification, les risques identifiés avec leurs plans B, les prochaines étapes et la conclusion.

1. Où nous en sommes

Les trois premières semaines ont surtout servi à comprendre le sujet en profondeur : la structure Half-Edge canonique, la logique de reconstruction des demi-arêtes jumelles (twin), et la façon dont les parcours (faces, voisinage d'un sommet) se déduisent des quatre pointeurs imposés. Ce temps a été investi volontairement avant d'écrire le code d'édition : une structure mal comprise coûte beaucoup plus cher à corriger après coup.

État au 17 octobre, par rapport au diagramme de Gantt (section 5) :

Jalon	Dates prévues	État vérifié dans le dépôt
M0 — Préparation	21/09 – 25/09	Fait : dépôt GitHub, diagramme UML (Lucidchart), roadmap (LaTeX + carte mentale XMind), Gantt (GanttProject), projet CMake. Le critère « hello triangle » sera atteint avec la fenêtre OpenGL (M2a).
M1 — Parsing & graphe	28/09 – 15/10	En grande partie fait : lecture de fichiers 3D (Assimp, .obj/.off et autres), construction du Half-Edge à partir des listes de sommets et de faces, parcours, statistiques et vérification d'Euler. Reste : export de fichier, checkIntegrity(), tests unitaires.
M2a — Rendu 3D	16/10 – 26/10	Démarré : prototype de visualisation interactive (page HTML générée par le programme) et export SVG des demi-arêtes. Reste : fenêtre OpenGL/GLFW, rendu filaire et ombré, caméra orbitale.
M2b → Rendu final	28/10 – 24/12	Pas encore commencés, conformément au planning.

Ce que nous pouvons montrer aujourd'hui : un cube (8 sommets, 12 triangles, 36 demi-arêtes) dont la structure est construite, parcourue et vérifiée (caractéristique d'Euler V − E + F = 2, aucune demi-arête sans jumelle, donc maillage fermé), avec deux sorties visuelles qui rendent la structure lisible : un schéma SVG des demi-arêtes et une vue 3D interactive.

Nous savons aussi charger un fichier de maillage (.obj, .off) et l'afficher en rendu ombré avec les arêtes visibles. La figure 1 montre deux cas de test utiles pour la vérification : un tore et une sphère .

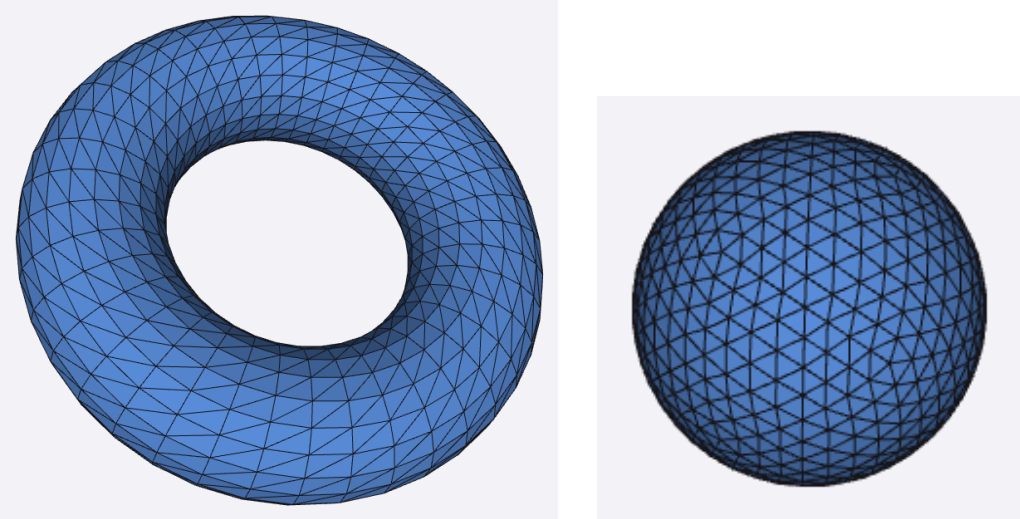
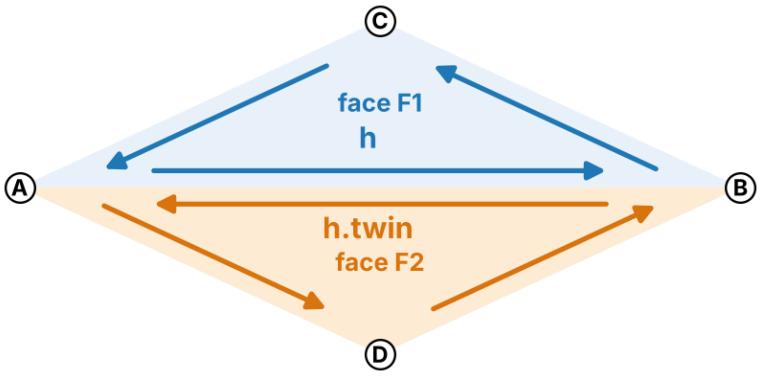


Figure 1 — Un tore (à gauche) et une sphère (à droite), chargés depuis des fichiers de maillage et affichés en rendu ombré avec arêtes visibles.

2. Comprendre le Half-Edge

Un maillage triangulaire est un graphe plongé dans R^3. La structure Half-Edge découpe chaque arête en deux demi-arêtes orientées, une par face adjacente. Chaque demi-arête ne stocke que quatre pointeurs, comme l'impose le cahier des charges :

- **target : le sommet d'arrivée (le cahier des charges l'appelle vertex) ;**
- **face : la face située à sa gauche ;**
- **next : la demi-arête suivante dans la même face, en sens anti-horaire ;**
- **twin : la demi-arête opposée, qui borde la face voisine (nulle sur un bord du maillage).**




```
          h = demi-arête A→B
    h.target = B   (sommet d'arrivée)
      h.face   = F1  (face à gauche)
h.next   = B→C (même face, sens anti-horaire)
h.twin   = B→A (demi-arête opposée, face F2)
```

Figure 2 — Deux faces partageant l'arête AB : h et son twin.

Conséquence importante de ce choix minimal : l'origine d'une demi-arête n'est pas stockée. On la retrouve en prenant la cible de la demi-arête précédente dans la face (en bouclant sur next). Pour un triangle, c'est simplement h->next->next->target.

2.1 Exemple sur notre cube

Nos 12 triangles donnent 36 demi-arêtes. La face f0 (sommets 0, 3, 2) est constituée de trois demi-arêtes qui bouclent par next :

Demi-arête	Origine → cible	Face	next	twin (face voisine)
h0	v0 → v3	f0	h1	h29 (f9)
h1	v3 → v2	f0	h2	h23 (f7)
h2	v2 → v0	f0	h0	h3 (f1)

Tableau 1 — Les demi-arêtes de la face f0 du cube (valeurs produites par notre programme).

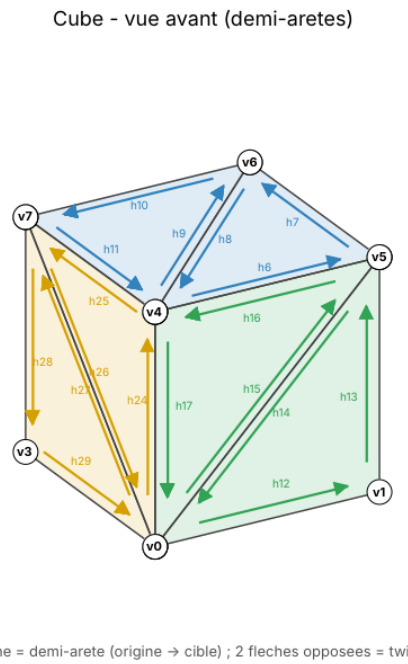


Figure 3 — Vue avant du cube, générée par notre programme : une flèche par demi-arête, deux flèches opposées pour une arête (twins).

2.2 Algorithme de construction

À partir de la liste des points et de la liste des faces (indices de sommets en sens anti-horaire) :

- **allouer d'un coup tous les sommets, faces et demi-arêtes (le nombre total de demi-arêtes est la somme des tailles des faces) ;**
- **pour chaque face et chaque côté (a → b), câbler target, face et next, puis mémoriser la demi-arête dans une table associative indexée par (a, b) ;**
- **à chaque insertion, chercher (b, a) : si elle existe, les deux demi-arêtes deviennent jumelles ; sinon l'arête est au bord et twin reste nul ;**
- **si (a, b) existe déjà, l'arête orientée est dupliquée : le maillage n'est pas orientable ou pas manifold, ce que nous détectons.**

2.3 Parcours du voisinage d'un sommet

La règle clé est : à partir d'une demi-arête sortante h d'un sommet v, la suivante autour de v est h->twin->next. Sur un sommet intérieur, la boucle se referme. Sur un sommet de bord, le tour est ouvert : on

recule d'abord jusqu'à la première demi-arête (via la précédente, puis son twin), puis on avance jusqu'à rencontrer un twin nul. Cette logique est déjà implémentée (fonction sortantes).

3. Architecture logicielle

3.1 Diagramme UML de l'existant

Le diagramme ci-dessous reflète le code actuellement dans le dépôt (halfedge.hpp et lecture.hpp). Le diagramme Lucidchart du dépôt reste la référence éditable ; celui-ci est généré à partir du code pour vérifier que les deux concordent.

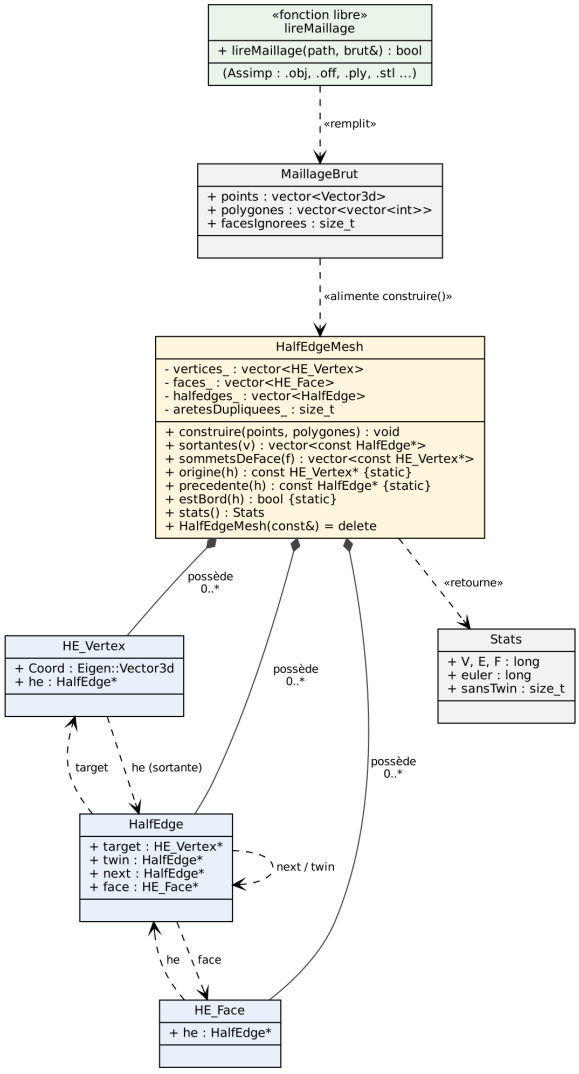


Figure 4 — Classes existantes. Losange plein : possession ; flèche pointillée : pointeur ou dépendance.

3.2 Stratégie de gestion mémoire

- **Un seul propriétaire. HalfEdgeMesh possède tous les sommets, faces et demi-arêtes dans trois std::vector ; les structures du cours ne portent que des pointeurs bruts non propriétaires, ce qui correspond à la contrainte des quatre pointeurs.**

- **Pas de réallocation après câblage. Les vecteurs sont dimensionnés une seule fois avant de créer les pointeurs, donc les pointeurs restent valides.**
- **Copie interdite. Une copie contiendrait des pointeurs vers le maillage d'origine ; le constructeur et l'affectation par copie sont supprimés (= delete).**
- **Lecture séparée de la topologie. lireMaillage remplit une structure intermédiaire (points + polygones), puis construire() câble le Half-Edge : le lecteur de fichiers ne connaît pas la structure.**

3.3 Architecture prévue (jalons 2 à 4)

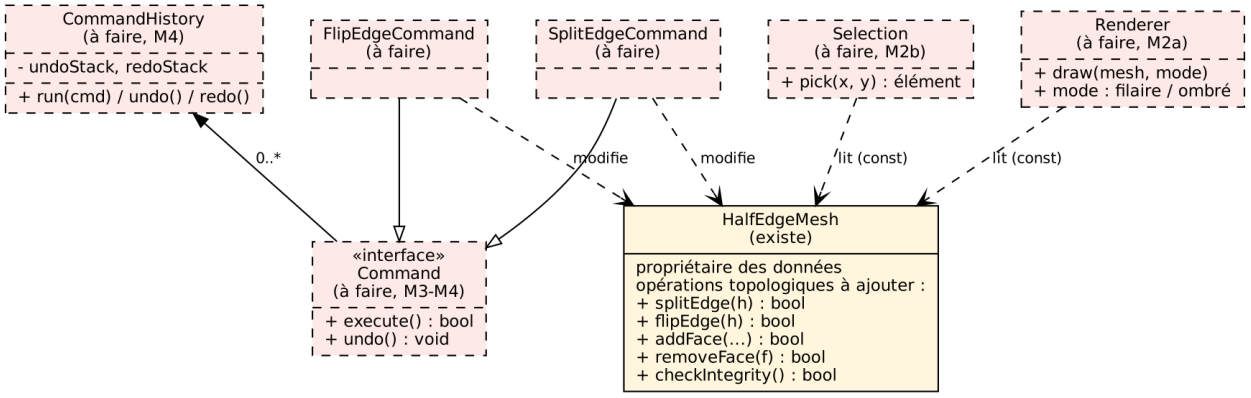


Figure 5 — Classes à venir (en pointillés rouges) : séparation données / rendu / interaction, et patron Command pour l'annulation.

Le rendu et la sélection ne font que lire le maillage (accès const). Toute modification passe par une commande (découpe d'arête, bascule d'arête, etc.) enregistrée dans un historique, ce qui donne l'annulation et le rétablissement au jalon 4 sans refonte.

4. Travail réalisé dans le code

Fichier	Contenu
halfedge.hpp	Structure Half-Edge (HE_Vertex, HE_Face, HalfEdge) et conteneur HalfEdgeMesh : construction, origine, précédente, test de bord, sommets sortants, sommets d'une face, statistiques (V, E, F, Euler, demi-arêtes de bord). Tolère les bords ; compte les arêtes dupliquées.
lecture.hpp	Lecture de fichiers 3D avec Assimp : triangulation, soudure des sommets dupliqués par position (tolérance 10^{-5}), rejet des faces dégénérées.
structure.cpp	Prototype autonome (sans dépendance) : même structure avec un index binaire (trie) sur les couples (origine, cible) pour retrouver les twins, affichage console, export SVG et HTML interactif. C'est lui que compile CMakeLists.txt aujourd'hui.
CMakeLists.txt	C++17, dépendances déclarées : Eigen, OpenGL, GLFW, Assimp.
roadmap_halfedge.tex, roadmap_mentale.jpeg, Gant.jpeg, uml diagram	Roadmap , carte mentale XMind, diagramme de Gantt (GanttProject), lien vers l'UML Lucidchart.

Vérification. Nous avons compilé et exécuté structure.cpp : le cube donne V = 8, E = 18, F = 12, soit V − E + F = 2 et zéro demi-arête sans jumelle. Une même structure est donc câblée de deux façons (table associative ou trie) ; nous les comparerons et n'en garderons qu'une.

4.1 Points à consolider (identifiés par relecture)

- **Deux implémentations coexistent (structure.cpp et halfedge.hpp) avec une convention différente pour le pointeur du sommet : « demi-arête entrante » dans l'une, « sortante » dans l'autre. Il faut unifier avant le jalon 3.**
- **lecture.hpp et halfedge.hpp ne sont pas encore appelés depuis le programme principal : le chargement d'un fichier .obj/.off jusqu'au Half-Edge reste à brancher de bout en bout.**
- **Pas encore d'export de fichier, de checkIntegrity() ni de tests unitaires : ils clôturent le jalon 1.**
- **Le rendu actuel est une page HTML générée ; la fenêtre OpenGL/GLFW reste à écrire (jalon 2a).**

5. Planification

Le planning a été établi avec GanttProject (début le 21 septembre 2026). Le critère « terminé » de chaque jalon est vérifiable, ce qui permet de décider sans débat quand basculer sur le plan B.

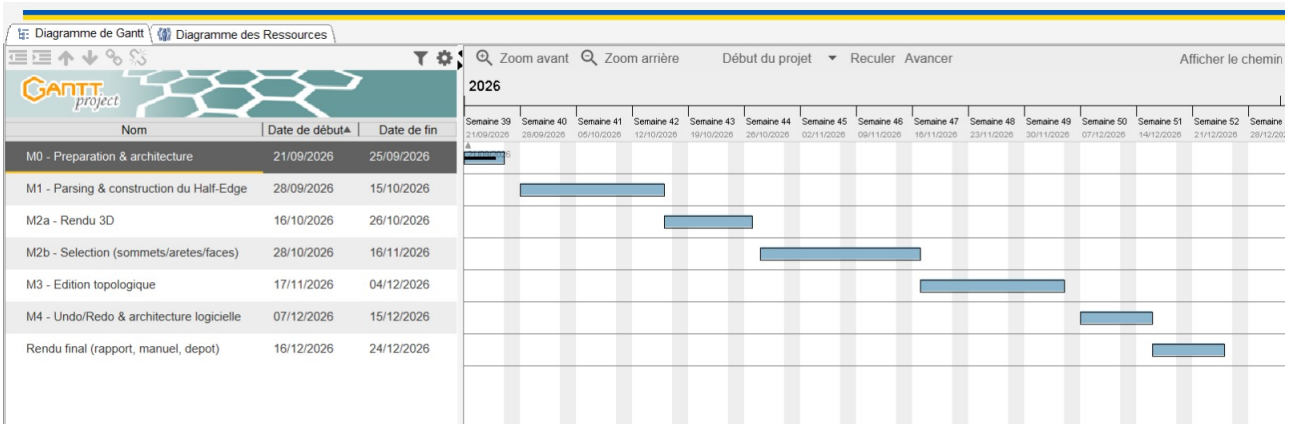


Figure 6 — Diagramme de Gantt du projet.

Dates	Jalon	Livrable	Critère « terminé »
21/09–25/09	M0	Dépôt, UML, roadmap, CMake	Un « hello triangle » s'affiche ; bibliothèque graphique choisie
28/09–15/10	M1	Import/export, Half-Edge, checkIntegrity(), tests	Import → export → import sans perte sur tétraèdre, cube, maillage réel
16/10–26/10	M2a	Rendu filaire et ombré, caméra orbitale	Un maillage chargé est visible et manipulable
28/10–16/11	M2b	Sélection sommets, arêtes, faces	Le clic sélectionne le bon élément
17/11–04/12	M3	Création de face, découpe et bascule d'arête, suppression	checkIntegrity() passe sur cas intérieurs et de bord
07/12–15/12	M4	Undo/Redo, séparation données/rendu/interaction	20 opérations défaites puis refaites, sans plantage
16/12–24/12	Rendu	Rapport LaTeX, manuel, dépôt propre	Un tiers compile et lance le projet

6. Risques et plans B

Pour chaque risque, une date de bascule est fixée à l'avance, rattachée à la fin d'un jalon : si le critère n'est pas atteint, on passe au plan B sans discussion. Les trois premières lignes viennent de notre relecture du code et sont les plus concrètes.

Risque	Plan A	Bascule	Plan B
Pointeurs invalidés quand les opérations ajoutent des éléments (la découpe d'arête crée un sommet, des demi-arêtes et des faces : un vector peut se réallouer)	Réserver la capacité à l'avance ; ou passer aux indices plutôt qu'aux pointeurs	Début M3 (17/11)	Stocker les éléments dans std::deque ou std::list, dont les adresses restent stables
Deux implémentations divergentes (convention du pointeur de sommet)	Garder halfedge.hpp, y reporter les tests de structure.cpp	Fin M1 (15/10, dépassée : à traiter en priorité)	Garder structure.cpp, plus simple, et y ajouter le bord
Soudure des sommets par position (tolérance 10^{-5}) qui fusionne des sommets distincts sur de très petits modèles	Tolérance proportionnelle à la taille du modèle	Fin M1	Fichiers simples d'abord ; option de désactiver la soudure
Bibliothèque graphique	OpenGL + GLFW	Fin M0 (25/09)	SFML ou plus simple à installer
Reconstruction des twin	Table associative (origine, cible) → demi-arête	Fin M1 (15/10)	Refuser les arêtes non-manifold ; twin nul sur les bords
Sélection par lancer de rayon	Ray casting 3D	Fin M2b (16/11)	Picking par couleur (rendu hors écran) ou sélection par indice
Cas de bord des opérations	Gérer bord et intérieur	~26/11	Intérieur d'abord ; l'opération renvoie false si les préconditions manquent, sans corrompre le maillage
Suppression d'un sommet	Réparation locale (remplir le trou)	Fin M3 (04/12)	Supprimer les faces incidentes, ou n'autoriser que les sommets isolés
Undo/Redo	Commande avec opération inverse	Fin M4 (15/12)	Instantané du maillage avant chaque opération
Retard global	Périmètre complet	Fin M3 (04/12)	Réduction du périmètre (ci-dessous)

Périmètre sacrifiable, dans cet ordre

- **Indispensable : M1, M2, découpe d'arête, bascule d'arête, rapport.**
- **Souhaitable : création de face, suppression, undo/redo.**
- **Bonus : extensions géométriques, second format de fichier, extension originale.**

Avant un bloc risqué (sélection, suppression de sommet), nous faisons un prototype de un à deux jours au maximum : s'il échoue, le coût est faible et le plan B est déjà prêt.

7. Prochaines étapes (jusqu'au 26 octobre)

- **Choisir l'implémentation de référence et unifier la convention du pointeur de sommet.**
- **Brancher de bout en bout : fichier .obj/.off → lireMaillage → construire → statistiques.**
- **Écrire checkIntegrity() (cycles next, symétrie des twin, cohérence face et sommet) et les premiers tests sur tétraèdre, cube et un maillage réel.**
- **Ajouter l'export .obj et vérifier le cycle import → export → import.**
- **Ouvrir une fenêtre OpenGL/GLFW avec rendu filaire, puis ombré et caméra orbitale (jalon 2a).**
- **Écrire la section LaTeX correspondante et ajouter des captures d'écran au dépôt.**

8. Conclusion

À mi-parcours, le noyau du projet est en place : la structure Half-Edge est construite à partir de listes de sommets et de faces, parcourue (faces, voisinage d'un sommet, bords) et vérifiée sur un maillage fermé (cube : V − E + F = 2, aucune demi-arête sans jumelle). Nous savons aussi charger un fichier de maillage et l'afficher (figure 1). Le temps passé à comprendre la structure nous donne des bases solides.

Avant le jalon 3, il reste à unifier les deux implémentations, écrire checkIntegrity(), l'export de fichier et les tests unitaires, puis à réaliser le rendu interactif complet (filaire, ombré, caméra, sélection). Le planning daté et le registre de plans B nous permettent de décider vite en cas de retard ; prochaine échéance : fin du jalon M2a, le 26 octobre.

Le principal défi sera le jalon 3 (découpe et bascule d'arête, suppression) : l'ajout d'éléments doit préserver la validité des pointeurs et les invariants de la structure, ce que checkIntegrity() et les commandes annulables (patron Command) doivent garantir.

9. Sources

[1] Claude, assistant d'IA développé par Anthropic (claude.ai) : aide apportée pour la partie rendu OpenGL.

[2] Dave Carrigg, « Nav Meshes Graphs and Half Edges », vidéo YouTube, 23 septembre 2020 (maillages de navigation, graphes et demi-arêtes). https://youtu.be/w5KOFgfx0CA

[3] Victor Gordan, « OpenGL Tutorial 12 - Mesh Class », vidéo YouTube, 26 mars 2021 (construction d'une classe Mesh en OpenGL). https://youtu.be/NUZF_5RKfS4

Les sources du cours : Ms_SoftEng_Labs1 HowToPassParameters

Annexe — Carte mentale de la roadmap

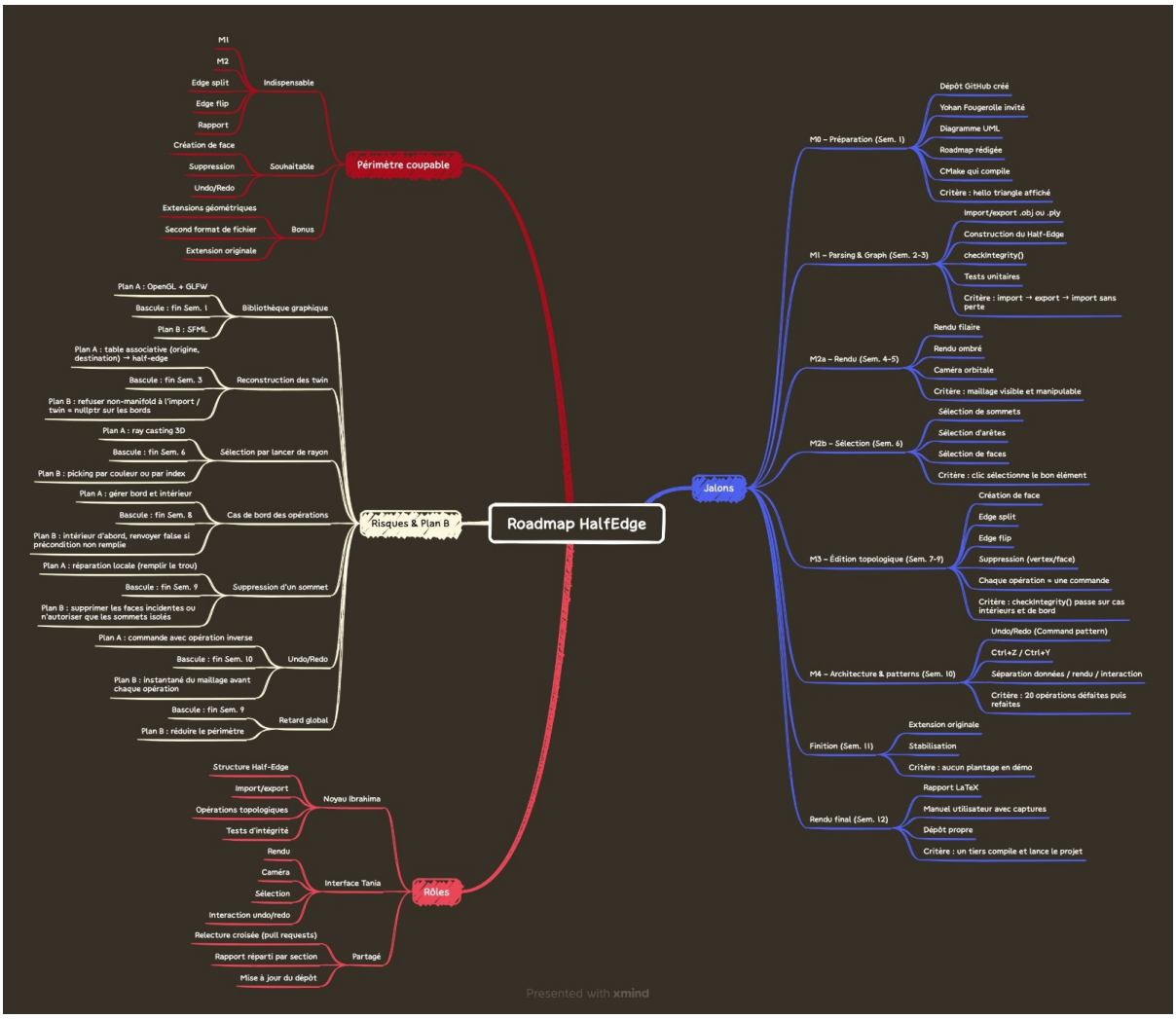


Figure 7 — Roadmap HalfEdge (XMind) : jalons, risques et plans B, rôles, périmètre sacrifiable.

UML éditable : lien Lucidchart dans le fichier « uml diagram » du dépôt.